

Atividade — Aula 10

TechFood: Tela de Cadastro de Pratos

Aluno(a): Andrew Clarck

Turma: 3ºB

Data: 08/06/2026

Objetivo da Atividade

Partindo do projeto TechFood entregue na Aula 9, construa a tela de cadastro de pratos (cadastro.html, cadastro.js e cadastro.css), seguindo o mesmo padrão de estilização e de estrutura de arquivos já existente no projeto. A tela deve cadastrar um novo prato e fazer com que ele apareça, de forma organizada, na tela principal (cardápio).

Para o envio da imagem do prato, escolha UMA das estratégias apresentadas nos slides da Aula 10 e implemente-a no seu sistema.

O Que Entregar

1. O projeto TechFood com a tela de cadastro funcionando (link do repositório ou pasta compactada).
2. Este documento preenchido, indicando a estratégia escolhida e como você a aplicou.

1. Estratégia de Imagem/Upload Escolhida

Foi utilizada a estratégia de **upload de arquivos com Multer**, onde a imagem do produto é enviada juntamente com os demais dados do formulário através de um objeto FormData.

Ao cadastrar um prato, a imagem é enviada para o servidor, armazenada na pasta de uploads e o caminho do arquivo é salvo no banco de dados. Posteriormente, esse caminho é utilizado para exibir a imagem no cardápio.

2. Como Você Implementou no Seu Sistema

No arquivo `cadastro.js`, foi criado um evento para interceptar o envio do formulário. Quando o usuário clica em **Cadastrar Produto**, o script:

1. Impede o recarregamento da página com `event.preventDefault()`.
2. Obtém os dados preenchidos pelo usuário.
3. Cria um objeto `FormData`, que permite enviar texto e arquivos na mesma requisição.
4. Ajusta os campos necessários antes do envio:
 - Converte o preço para o formato aceito pelo banco de dados (0.00).
 - Converte o campo "Disponível" para `1(True)` ou `0(False)`.
5. Chama a função `criarProduto()` do arquivo `api.js`.

A função `criarProduto()` realiza uma requisição HTTP:

POST /produtos

enviando o `FormData` para o back-end.

No servidor, o Multer recebe a imagem, salva o arquivo na pasta de uploads e disponibiliza seus dados para a aplicação. Em seguida, o produto é validado e armazenado no banco de dados junto com o caminho da imagem.

Quando a página inicial é carregada, a função `buscarProdutos()` consulta a API (`GET /produtos`) e retorna todos os produtos cadastrados. A função `renderizarCardapio()` percorre esses produtos e cria dinamicamente os cards do cardápio, exibindo nome, descrição, preço e imagem.

Assim, após um novo cadastro, o produto passa a estar disponível para ser exibido no cardápio.

3. Arquivos Criados ou Alterados

Arquivo	Alteração
<code>cadastro.html</code>	Criação do formulário de cadastro de produtos e campo para upload de imagem.
<code>cadastro.css</code>	Estilização completa da tela de cadastro, incluindo formulário e botão personalizado de upload.
<code>cadastro.js</code>	Implementação da lógica de envio do formulário, preview da imagem e integração com a API.
<code>api.js</code>	Criação da função <code>criarProduto()</code> para enviar os dados ao back-end.
<code>errorHandler.js</code>	Envio de erro em JSON do multer (imagem com tamanho acima do limite) para o front (tratamento de erros).

4. Dificuldades e Como Resolveu

- **Campo "Disponível":** o checkbox enviava os valores `"true"` e `"false"`, mas o banco de dados esperava valores inteiros. A solução foi converter os valores para `"1"` e `"0"` antes do envio.
 - **Campo de preço:** o usuário digitava valores com vírgula (`16,90`), porém o banco de dados aceitava apenas ponto decimal (`16.90`). Foi necessário realizar a conversão antes de enviar os dados.
 - **Erro de upload de imagem:** algumas imagens ultrapassavam o limite de tamanho configurado no Multer (2 MB), gerando erro de upload. O problema foi identificado através dos logs do servidor e resolvido utilizando imagens menores e implementando tratamento adequado para erros de upload.
 - **Experiência do usuário:** o componente padrão de upload de arquivos possuía uma aparência simples. Para melhorar a interface, foi criado um botão personalizado e uma pré-visualização da imagem selecionada antes do envio do formulário.
-

Checklist Final (autoavaliação)

- x Criei cadastro.html, cadastro.js e cadastro.css seguindo o padrão do projeto.
- x O formulário valida os campos antes de enviar.
- x O cadastro envia os dados ao back-end (POST /produtos) com sucesso.
- x O prato cadastrado aparece na tela principal de forma organizada.
- x Preenchi e anexei este documento.